

Sermon Archive / Transcription Project

Comprehensive summary of the project discussions, decisions, research findings, and recommended build path

Prepared for Evan Thompson - June 8, 2026

Core commitment: this project and the trading bot are the two active projects. Everything else waits unless it directly supports one of these two efforts.

1. Executive Summary

The sermon archive project is intended to preserve and organize Pastor Jay Thompson's sermons by turning sermon recordings into clean audio, full transcripts, structured summaries, scripture references, searchable metadata, and eventually a long-term searchable archive. The project began as a simple question about extracting audio from recorded sermon videos and producing documents of the preached dialogue. It grew into a serious, practical archive system with two tracks: future automation for new OBS recordings and back-catalog recovery for older Facebook Live and YouTube Live sermons.

The project is realistic for a one-person developer if it is built in phases. The first version should not try to automate Facebook downloads, build a dashboard, or create a professional church library. The first version should prove one thing: one sermon file can become audio, transcript, summary, metadata, and a catalog record.

2. Project Purpose and Scope

The purpose is not merely to create transcripts. The goal is to preserve a meaningful preaching archive that can be searched, read, studied, backed up, and expanded over time.

Primary goals:

- Preserve past and future sermons in a reliable digital archive.
- Extract audio from recorded sermon videos when needed.
- Transcribe sermons into full text.
- Generate structured sermon records including title, preacher, date, scripture, themes, summary, outline, key quotes, and call to action.
- Catalog sermons so they can be searched and retrieved later.
- Keep the workflow realistic for one person with no local technical support.
- Use low-cost APIs or local tools where practical.

Out of scope for the first version:

- A polished public website or dashboard.
- Fully automated Facebook scraping or fragile social-media automation.
- Complex microservices, container orchestration, or cloud databases.
- Vector search or semantic search before plain transcript processing works.
- Trying to process the entire historical archive before testing with a few representative sermons.

3. Main Decisions Made So Far

Decision	Current direction	Reason
Two active projects	Sermon Archive and Trading Bot only	Limits distraction and increases the chance of actually finishing both.
Future recordings	Use OBS recordings as the main future input	OBS is already part of the church workflow and can record clean audio tracks.
Microphone setup	Dedicated microphone is available	This should greatly improve transcription quality compared with room audio.
Past sermons	Recover from Facebook Live and YouTube Live	This preserves as much of the preaching archive as possible.
Video storage	Treat video as temporary or optional; keep audio as the working archive	Audio is smaller, easier to process, and sufficient for transcription.
First milestone	Process one test sermon end-to-end	Proves the pipeline before bulk downloading or overbuilding.
Research tool	Use Gemini Deep Research for high-volume planning research	Gemini's research limits appear more predictable for the planning stage.

4. Desired Output Per Sermon

The archive should preserve the sermon and also make it understandable. The safest design separates preservation outputs from interpretation outputs.

Preservation layer:

- Original source file if kept
- Extracted audio
- Raw transcript
- Cleaned transcript

Understanding layer:

- Sermon title or AI-suggested title
- Date preached
- Speaker/preacher
- Main scripture passage
- Other Bible references
- Main topic/theme
- Short summary
- Detailed outline
- Key quotes
- Call to action or altar call theme

- Source file
- Transcript confidence/issues

5. Recommended File Output

A clean archive should be easy for a human to browse and easy for Python scripts to process. The research strongly supports a folder-first system with one folder per sermon, JSON sidecar metadata, and later a SQLite search database.

Example folder layout:

```
sermon_archive/ incoming/ facebook/ youtube/ obs/ archive/ 2026/ 2026-06-07_Suggested_Title/
2026-06-07_Suggested_Title.mp3 2026-06-07_Suggested_Title_transcript_raw.txt
2026-06-07_Suggested_Title_transcript_clean.txt 2026-06-07_Suggested_Title_summary.md
2026-06-07_Suggested_Title.json 2026-06-07_Suggested_Title.docx catalog/ sermon_catalog.csv
sermon_archive.db
```

Naming convention:

```
YYYY-MM-DD_Source_Or_SermonSlug.ext
```

Examples: 2024-02-04_facebook_sermon.mp3, 2026-06-07_obs_The_Course_Of_This_World.txt,
2026-06-07_New_Life_Church.json

6. Future Sermons: OBS Workflow

Future sermons should be the easiest part because the church already uses OBS and has a dedicated microphone. The ideal future setup is to record the normal full service audio and, if possible, a separate pastor-mic-only track. That pastor-only track would produce cleaner transcripts because it avoids music, audience noise, and side conversations.

Recommended future pipeline:

```
OBS records service -> recording lands in incoming/obs/ -> script extracts pastor mic audio or the
best available audio track -> transcription API or local model creates transcript -> AI analysis
creates summary, outline, scripture list, quotes, and call to action -> system writes transcript,
summary, JSON metadata, and catalog/database entry -> processed file is moved to archive/ or
processed/
```

7. Past Sermons: Facebook Live and YouTube Live Recovery

The back-catalog is the harder part because old sermons may be buried in Facebook or YouTube, may have inconsistent titles, and may not have clean metadata. The recommended approach is manual download first, automated processing second. Do not begin by trying to automate Facebook scraping.

Recommended back-catalog recovery plan:

- Download or export a small number of old sermons first: one older Facebook sermon, one newer Facebook sermon, and one YouTube sermon.
- If only video downloads are available, download the video temporarily, extract audio, verify the transcript, then decide whether to keep or delete the video.
- Keep audio files as the permanent working archive because they are smaller and sufficient for transcription.
- Use a thumb drive or external SSD for collection and transfer, but avoid making the thumb drive the only copy.

- Apply consistent filenames immediately, even if the title is unknown.

8. Audio-Only Strategy

Audio-only is the smart default for the working archive. The video is useful as a source, but the transcription system does not need the picture. Keeping audio reduces storage, processing time, upload time, and backup costs.

Asset	Keep long term?	Reason
Extracted audio	Yes	Primary archival media for transcription and listening.
Transcript	Yes	Searchable and readable record of the sermon.
Summary and metadata	Yes	Makes the archive useful for study and retrieval.
Original video	Optional	Useful for preservation, but storage-heavy. Treat as temporary unless storage is cheap or the video is important.

9. Online API Cost Discussion

For Sunday-only usage, transcription API cost was estimated as very low. A one-hour sermon once per Sunday would likely cost under a few dollars per month with common low-cost transcription APIs. The earlier rough estimate for OpenAI transcription was roughly \$0.72 to \$2.16 per month for four 60-90 minute sermons, depending on model choice. The uploaded transcription research also compared several services and suggested that some providers may include enough free monthly transcription for a weekly sermon workflow.

Important caution: pricing and free tiers change. Before coding against a provider, verify the current pricing page and file-size limits.

10. Research Findings from the Uploaded Reports

Three research documents were added to the project source material: Sermon Archive System Architecture, Sermon Transcription Service Comparison, and AI Sermon Metadata Extraction System. These reports support a staged, local-first, low-complexity approach.

Architecture research findings:

- Use a local-first architecture because it is cheaper, easier to control, and suitable for one person.
- Use folders for assets and JSON sidecar files for sermon-specific metadata.
- Use SQLite with FTS5 later for fast transcript search without running a separate search server.
- Avoid hosted cloud databases, Elasticsearch, microservices, and complex dashboards early.
- Back up locally and offsite; Backblaze B2 plus rclone was recommended as a future low-cost backup path.

Transcription research findings:

- OBS recordings from a dedicated microphone should be far cleaner than old compressed Facebook/YouTube live audio.
- Long sermons create special challenges: pauses, emotional cadence, accents, music, and audience noise.
- OpenAI Whisper-style APIs can be accurate, but file-size limits and silence hallucinations can complicate long sermons.
- AssemblyAI, Deepgram, Speechmatics, Gladia, Azure, Google, and local Whisper/WhisperX were all compared.

- For a fast Version 1, a clean API with strong long-file handling may be easier than maintaining a local GPU pipeline.
- For a large back-catalog, local processing or generous free/low-cost batch transcription becomes more attractive.

Metadata extraction research findings:

- JSON is the preferred format for structured sermon metadata because it is easy to validate and works well with modern structured-output LLM methods.
- Pydantic-style schemas can enforce field names, data types, and missing-value rules.
- Key quotes should be verified against the raw transcript so the AI cannot invent or polish quotes.
- Scripture references should be validated with deterministic Python libraries such as pythonbible or python-scriptures rather than trusting the AI alone.
- Long transcripts may eventually need chunking or map-reduce processing, but Version 1 should keep this simple.

11. Recommended Technical Architecture

The recommended architecture is a simple sequential Python pipeline. This is intentionally boring. Boring is good here because a one-person archive project needs reliability more than cleverness.

Version 1 pipeline:

```
Input file in incoming/ -> extract audio with ffmpeg when source is video -> send audio to
transcription service or local model -> save raw transcript as TXT -> run AI metadata extraction
-> save summary.md and sermon_record.json -> append/update sermon_catalog.csv -> later: insert
into SQLite database
```

Practical Version 1 stack:

Layer	Recommended simple choice	Later upgrade
Input	Manual file drop into incoming/	Automated OBS folder watcher
Audio extraction	ffmpeg	Track-specific extraction from OBS multi-track recordings
Transcription	API for simplicity or faster-whisper for local	Provider comparison, fallbacks, batch queue
Metadata	Structured JSON with strict fields	Pydantic validation and retry loop
Catalog	CSV and per-sermon JSON	SQLite with FTS5
Search	File search / basic script	SQLite FTS5 dashboard, then semantic search

12. Metadata Schema Draft

Every sermon should eventually have a machine-readable JSON record. The schema should allow unknown values instead of forcing the AI to guess. This preserves archive integrity.

```
{ "date_preached": "YYYY-MM-DD or null", "preacher": "Pastor Jay Thompson or null",
"sermon_title": "Actual or AI-suggested title", "title_was_suggested": true, "source": "OBS |
Facebook Live | YouTube Live | Unknown", "main_scripture": "Romans 8:28 or null",
"other_scriptures": ["James 1:2-4", "Psalm 46:1"], "main_theme": "Short theme label", "summary":
"One-paragraph summary", "outline": [ {"level": 1, "point": "Main point", "takeaway":
"Application"} ], "key_quotes": ["Exact quote from transcript"], "call_to_action": "Main response
urged by the sermon", "transcript_confidence": "Good | Fair | Poor | Needs Review", "issues":
["Music at beginning", "Unclear phrase around 18:42"], "files": { "audio": "relative/path.mp3",
```

```
"raw_transcript": "relative/path.txt", "summary": "relative/path.md" } }
```

13. One-Person Project Rules

Because there is no local technical help available, the process has to be designed around focus, repeatability, and recovery from mistakes.

- Build one working piece at a time.
- Do not automate downloads until manual download plus processing works.
- Do not build a user interface until the command-line pipeline works.
- Do not process 100 sermons until 3 representative sermons work.
- Keep source files, transcripts, and metadata organized from the start.
- Commit meaningful project updates once the repository exists.
- Maintain a project README and teaching_points.md so lessons and commands are not lost.

14. Phased Roadmap

Phase	Goal	Done when
Phase 0 - Planning	Define scope, outputs, and source research	This summary exists and the research docs are saved.
Phase 1 - Manual Proof	Process one sermon file manually	One sermon produces audio, transcript, summary, and JSON.
Phase 2 - Metadata	Add title, scriptures, summary, outline, quotes, call to action	Metadata extraction is consistent enough to review.
Phase 3 - Catalog	Create sermon_catalog.csv and/or SQLite index	Processed sermons are searchable by date, source, title, and transcript text.
Phase 4 - Back Catalog	Batch process old Facebook/YouTube audio	A folder of downloaded sermons can be processed safely without duplicates.
Phase 5 - OBS Automation	Connect future Sunday recordings to pipeline	A new OBS recording can be processed with minimal manual steps.
Phase 6 - Search/Interface	Optional dashboard or local search tool	User can search sermons by word, scripture, topic, or date.

15. Immediate Next Steps

The next move should be practical, not theoretical. We have enough research to start small.

- Create or confirm the project folder/repository name: sermon_archive.
- Choose three test files: one recent OBS recording, one older Facebook Live sermon, and one YouTube Live sermon if available.
- Decide whether Version 1 transcription uses an online API or local faster-whisper. For speed and simplicity, an API is probably easier at first.
- Create the initial folder structure: incoming/, archive/, transcripts/, catalog/, scripts/.

- Build the first script: take one local video/audio file and produce audio.mp3 plus transcript_raw.txt.
- Only after that, add AI summary and metadata extraction.

16. Risks and Mitigations

Risk	Mitigation
Old videos are hard to download or missing	Start with a few test files and document the exact recovery process before bulk work.
Transcripts contain errors from music, echo, or pauses	Use dedicated mic audio for future sermons; flag transcript issues; review key passages manually.
AI invents scripture references or quotes	Validate scripture references with a parser and verify key quotes against the raw transcript.
Project becomes too large	Stick to the roadmap. No dashboard, scraping automation, or semantic search until core processing works.
Data loss	Use thumb drive/external drive plus computer copy, then add offsite backup later.

17. Source Notes

This PDF summarizes the planning conversation and incorporates the three uploaded research reports. Source documents used:

- AI Sermon Metadata Extraction System - supports structured JSON metadata, validation, scripture parsing, quote verification, and staged metadata extraction.
- Sermon Archive System Architecture - supports the local-first folder, JSON sidecar, SQLite FTS5, ffmpeg, faster-whisper, rclone, and backup architecture.
- Sermon Transcription Service Comparison - supports the comparison of transcription APIs, local Whisper options, costs, file limits, diarization, and long-sermon gotchas.

This is a planning summary, not a final technical specification. Prices, API limits, and provider features should be rechecked immediately before implementation.